

UNISAL

Centro Universitário Salesiano de São Paulo

SISTEMA DE CONTROLE DE ESTOQUE DE PRODUTOS PERECÍVEIS

Documento Arquitetural Completo

Engenharia de Software

Integrantes

Ana Beatriz de Oliveira Elias

Elisa da Silva Ortiz de Godoy

Larissa dos Santos Oliveira

Vinicius Gama Bittencourt

Lorena - SP

2026

1 Evolução do Diagrama de Classes

1.1 Diagrama de Classes – Versão Original

A versão original do diagrama de classes foi construída diretamente a partir dos requisitos funcionais levantados na Parte 1 do projeto. Ela representa a estrutura inicial do Sistema de Controle de Estoque de Produtos Perecíveis, contemplando as entidades principais sem a aplicação formal de padrões de projeto.

Classes identificadas na versão original

- **Pessoa:** representa os dados básicos dos indivíduos cadastrados no sistema, contendo atributos como nome, CPF, número de celular, e-mail, data de nascimento e residência. Possui operações de cadastro, consulta, edição, remoção e validação de CPF.

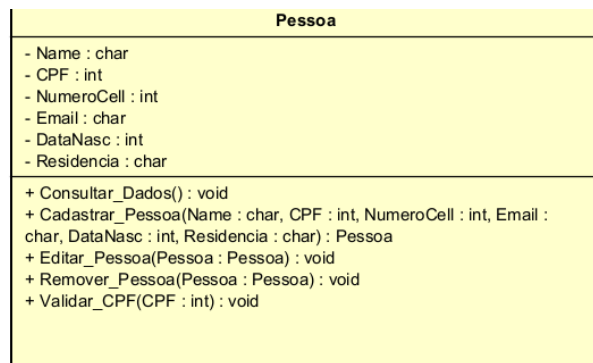


Figura 1: Diagrama: Classe Pessoa

- **Funcionário:** especialização da classe **Pessoa**, responsável pelo gerenciamento dos usuários internos do sistema. Possui atributos relacionados ao cargo, salário, situação e senha, além de operações de cadastro, edição e autenticação.

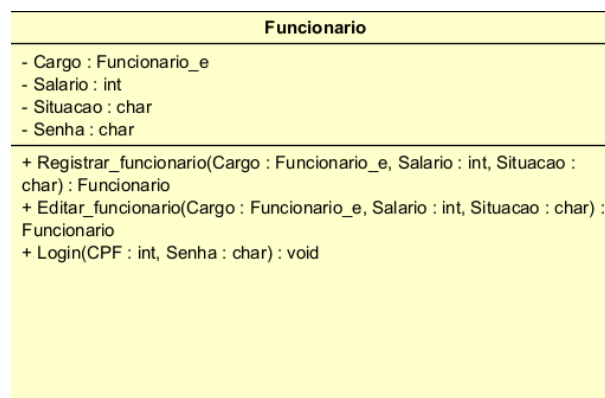


Figura 2: Diagrama: Classe Funcionário

- **Funcionario_e:** enumeração responsável pela definição dos cargos existentes no sistema, sendo composta pelos valores *Gerente* e *Estoquista*.

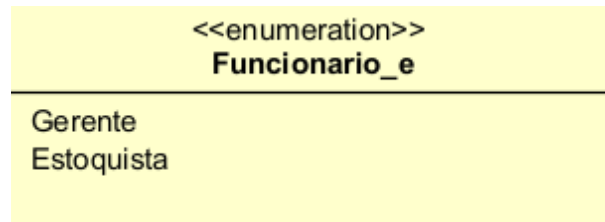


Figura 3: Diagrama: Classe Funcionário_e

- **Fornecedor:** representa as empresas responsáveis pelo abastecimento do estoque, armazenando informações como nome, CNPJ, empresa e logradouro. Também possui operações de cadastro, consulta, atualização e configuração de rotas.

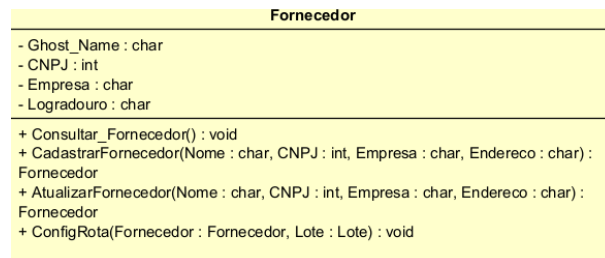


Figura 4: Diagrama: Classe Fornecedor

- **Produto:** representa os itens armazenados no estoque, contendo atributos como código e código de barras. Possui operações de cadastro, consulta e atualização.

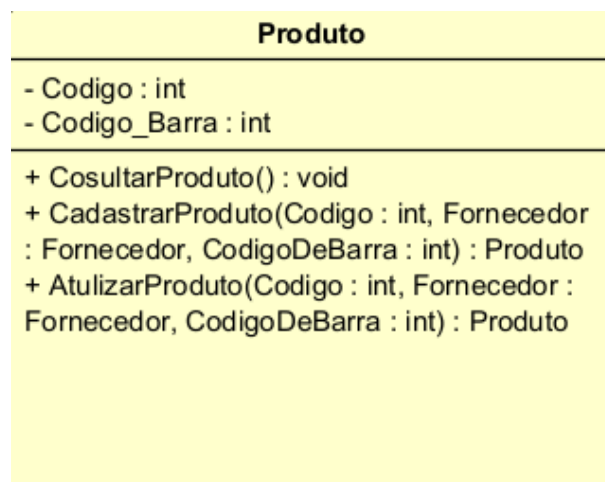


Figura 5: Diagrama: Classe Produto

- **Lote:** representa os lotes dos produtos armazenados, contendo informações referentes ao código, produto associado, validade, quantidade, data de recebimento e data de fabricação. Além disso, concentra operações de cadastro, consulta, controle de validade, alerta de quantidade, registro de perdas, criação de relatórios e visualização do prazo de giro dos produtos.

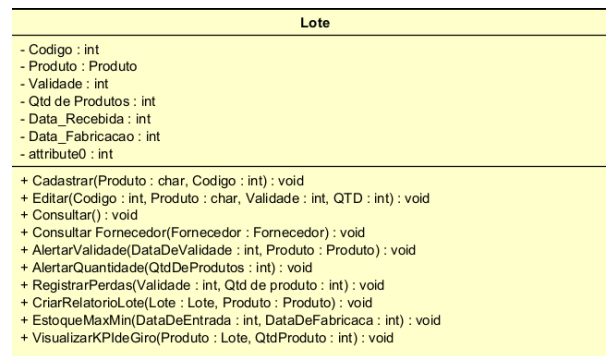


Figura 6: Diagrama: Classe Lote

Problemas identificados na versão original

- A classe **Lote** acumulava diversas responsabilidades relacionadas ao controle do estoque, monitoramento da validade, emissão de alertas, registro de perdas e geração de relatórios, caracterizando uma violação do Princípio da Responsabilidade Única (SRP).
- Não existia uma classe central responsável pelo gerenciamento das operações do estoque, dificultando a organização e a manutenção do sistema.
- As regras de controle dos produtos estavam fortemente acopladas às classes do domínio, reduzindo a flexibilidade da arquitetura.
- Os módulos externos precisavam acessar diretamente diversas classes para executar operações do sistema, aumentando o acoplamento entre os componentes.
- Não havia uma abstração para diferentes estratégias de gerenciamento e reposição dos produtos, tornando a lógica rígida e dificultando futuras extensões.
- A estrutura geral do sistema apresentava baixa modularidade, reduzindo a reutilização de código e dificultando a evolução da arquitetura.

1.2 Diagrama de Classes – Versão Evoluída

A versão evoluída incorpora os padrões de projeto do catálogo Gang of Four (GoF), com o objetivo de corrigir os problemas identificados e aumentar a manutenibilidade, extensibilidade e coesão do sistema.

Classes e interfaces adicionadas

- **FuncionarioFactory** (*<interface><FactoryMethod>*): define a interface para o método de fabricação de funcionários, estabelecendo o contrato para a aplicação do padrão Factory Method.
- **GerenteFactory** e **EstoquistaFactory**: classes de fábricas concretas que implementam a interface **FuncionarioFactory**. Elas encapsulam a lógica de instanciação

isolada, sendo responsáveis por criar e retornar os objetos específicos **Gerente** e **Estoquista**, respectivamente.

- **Gerente e Estoquista** (subclasses): especializações físicas criadas a partir da classe **Funcionario**. Substituem o uso antigo de enumeradores por herança pura, permitindo que cada tipo de funcionário possua comportamentos e regras de negócio isoladas, explorando o polimorfismo.

1.3 Principais Alterações Estruturais

A Tabela 1 apresenta as principais diferenças entre a versão original e a versão evoluída do sistema após a aplicação dos padrões de projeto.

Tabela 1: Comparativo entre a Versão Original e a Versão Evoluída

Aspecto	Versão Original	Versão Evoluída
Padrão de Criação de Objetos	Instanciação direta e acoplada no código cliente ou condicionada por flags de tipo.	Descentralização e isolamento da lógica de criação utilizando o padrão Factory Method (FuncionarioFactory).
Modelagem de Cargos / Papéis	Utilização de um atributo enumerado dentro de uma única classe genérica Funcionario.	Substituição do Enum por herança e especialização de classes com o surgimento das subclasses Gerente e Estoquista.
Tipagem de Dados (Segurança e Máscaras)	Atributos estruturados com tipos primitivos inadequados (ex: CPF, Telefone e Data de Nascimento como int; Nome e Senha como char).	Correção estrita e tipagem forte nos atributos de Pessoa e Funcionario, passando a utilizar String, double e Date.
Regras e Validações de Domínio	Inexistência de rotinas de validação de dados estruturadas na entidade que gerencia as informações.	Adição do método especialista <code>validarCPF(cpf : String) : boolean</code> diretamente na classe de domínio Pessoa.is
Assinatura de Métodos de Cadastro	Métodos com longas listas de parâmetros primitivos na assinatura	Simplificação e encapsulamento das operações de ciclo de vida (ex: <code>cadastrarPessoa() : void</code>), favorecendo o uso de objetos de transferência.

2 Documento Arquitetural – Aplicação dos Padrões GoF

Os cinco padrões aplicados foram escolhidos de forma que nenhum coincidissem com os padrões apresentados por outros grupos durante os seminários. Foram selecionados os padrões Factory Method, Singleton, Facade, Strategy e Template Method, aplicados estrategicamente aos domínios de Controle de Acesso e Gerenciamento de Estoque de Produtos Perecíveis.

2.1 Padrão 1: Factory Method [Criacional]

Problema Identificado

O sistema precisava criar diferentes tipos de Produto (perceíveis, secos e refrigerados) e Lote com configurações distintas. Na versão original, os objetos eram instanciados diretamente com `new` no código cliente, espalhando a lógica de criação por todo o sistema. Isso tornava impossível variar o tipo de objeto criado sem modificar cada ponto de instância, violando o Princípio Aberto/Fechado (OCP).

Solução Proposta

Aplicação do padrão Factory Method. A interface `FuncionarioFactory` declara o método `criarFuncionario() : Funcionario`; As classes concretas `GerenteFactory` e `EstoquistaFactory` implementam essa interface, assumindo a responsabilidade de instanciar e configurar os respectivos objetos especializados (Gerente e Estoquista). O código cliente passa a operar exclusivamente através do contrato das fábricas.

Justificativa Técnica

O Factory Method delega a decisão de qual classe concreta instanciar para as subclasses especialistas de criação, desacoplando o fluxo de execução principal da lógica de construção. No contexto de gestão de usuários, isso isola o processo de parametrização e controle de privilégios de acesso associados a cada cargo.

Por que usar o padrão?

O sistema gerencia múltiplos papéis de usuários com responsabilidades distintas. O Factory Method centraliza as regras de construção, elimina condicionais complexas de validação no cliente e permite a adição de novos cargos futuramente sem a necessidade de alterar ou quebrar o código existente.

Impacto Arquitetural

A lógica de criação foi encapsulada nas fábricas concretas. Componentes externos ou chamadores dependem unicamente da abstração `FuncionarioFactory`, elevando a exten-

sibilidade da arquitetura do módulo de identidade e autenticação do sistema

Vantagens

- Centraliza e encapsula as regras de construção;
- Permite adicionar novos tipos de produto sem modificar código existente;
- Facilita testes unitários;
- Elimina duplicação de código de validação.

Possíveis Limitações

- Introduce uma quantidade maior de classes e interfaces no ecossistema do projeto;
- Exige maior maturidade do código consumidor para depender de abstrações.

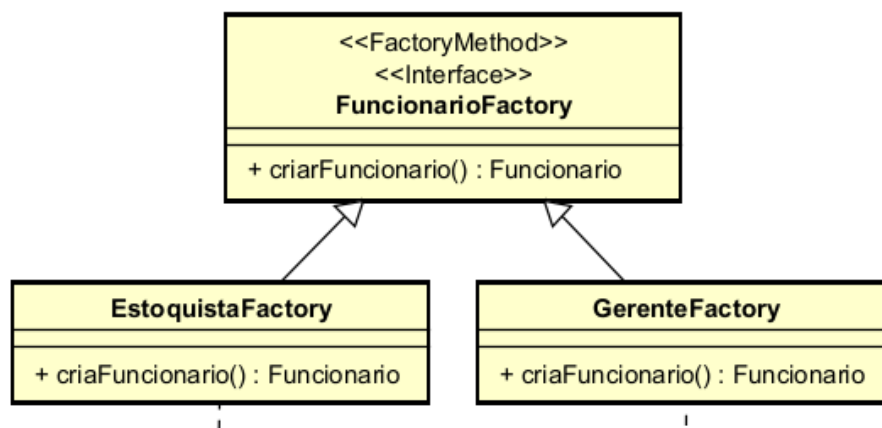


Figura 7: Diagrama:FuncionarioFactory

2.2 Padrão 2: Singleton [Criacional]

Problema Identificado

A classe *Estoque* é responsável pelo gerenciamento central das operações físicas do inventário, incluindo o armazenamento dos lotes e a execução das estratégias de reposição. Em uma implementação convencional, seria possível criar múltiplas instâncias dessa classe, fazendo com que diferentes partes da aplicação manipulassem conjuntos distintos de lotes em memória. Essa situação poderia causar inconsistências nos dados do inventário, duplicidade de registros e perda da integridade das informações armazenadas.

Solução Proposta

Foi aplicado o padrão Singleton à classe *Estoque*, restringindo sua instanciação a um único objeto em toda a aplicação. Para isso, o construtor `Estoque()` foi definido com

visibilidade privada e foi criado o atributo estático interno `instancia` : `Estoque`, responsável por armazenar a única instância existente da classe. O acesso global ao objeto é realizado exclusivamente por meio do método público `getInstance()` : `Estoque`.

Além disso, a classe mantém internamente:

`lotes` : `List<Lote>`, responsável pelo armazenamento dos lotes cadastrados; `estrategia` : `EstrategiaReposicao`, utilizada para definir a estratégia de reposição empregada pelo sistema.

As operações disponibilizadas pela classe incluem a adição, remoção e consulta de lotes, bem como a definição e execução da estratégia de reposição.

Justificativa Técnica

Em sistemas de controle de estoque, todas as operações relacionadas ao inventário devem atuar sobre uma única coleção de dados, garantindo que as movimentações sejam consistentes e centralizadas. A existência de apenas uma instância da classe `Estoque` assegura que todos os módulos da aplicação compartilhem a mesma lista de lotes (`lotes` : `List<Lote>`), preservando a integridade das informações e evitando estados divergentes em memória.

Por que usar o padrão?

A classe `Estoque` representa um recurso compartilhado e essencial do sistema. A utilização do padrão Singleton garante uma única fonte de informação para o estado atual do inventário, assegurando que todas as operações de inserção, remoção, consulta e reposição sejam realizadas sobre o mesmo objeto.

Impacto Arquitetural

Todos os subsistemas da aplicação passam a acessar o inventário por meio da chamada estática `Estoque.getInstance()`. Dessa forma, as operações `adicionarLote()`, `removerLote()`, `consultarLotes()`, `definirEstrategia()` e `executarReposicao()` atuam sempre sobre a mesma instância da classe, uniformizando o gerenciamento do estoque e evitando inconsistências de dados.

Vantagens

- Garante consistência do estado do inventário;
- Fornece acesso global controlado;
- Permite inicialização sob demanda;
- Serve de barreira contra instanciações acidentais em outras camadas.

Possíveis Limitações

- Pode dificultar testes unitários;
- Introduce um acoplamento implícito que pode ocultar falhas de arquitetura orientada a componentes.

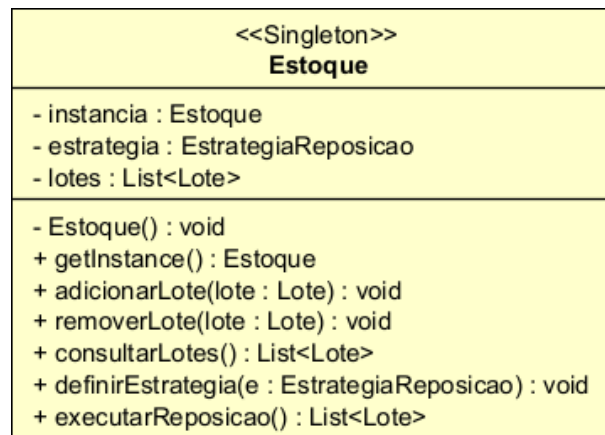


Figura 8: Diagrama:Singleton

2.3 Padrão 3: Facade [Estrutural]

Problema Identificado

O ecossistema de estoque é composto por um conjunto complexo de interações entre as entidades **Fornecedor**, **Produto** e **Lote**. Na modelagem original, os módulos consumidores externos necessitavam orquestrar e chamar de forma sequencial os métodos e ciclos de vida de cada uma dessas classes para concluir tarefas básicas (como cadastros cruzados), gerando alto acoplamento, código redundante e fragilidade a alterações internas.

Solução Proposta

Aplicação do padrão Facade por meio da classe **SistemaEstoque**. Essa classe atua como fachada do subsistema, centralizando operações complexas em métodos simplificados de alto nível para os clientes da aplicação, tais como **cadastrarProduto()**, **cadastrarFornecedor()**, **registrarLote()**, **consultarValidade()**, **executarFEF0()**, **gerarRelatorioValidade()** e **gerarRelatorioPerdas()**. Dessa forma, os consumidores do sistema interagem com uma interface única e simplificada, sem a necessidade de conhecer os detalhes das classes internas do domínio.

Justificativa Técnica

O padrão Facade reduz significativamente a complexidade operacional do sistema ao disponibilizar uma interface unificada que encapsula a colaboração entre as classes do domínio. Isso protege a integridade dos fluxos internos, diminui o acoplamento entre os

módulos clientes e o subsistema de estoque e torna mais simples a realização de modificações internas sem impactar as camadas externas.

Por que usar o padrão?

As operações relacionadas ao gerenciamento de produtos perecíveis envolvem diversas etapas e interações entre entidades do domínio. A fachada encapsula essas operações em métodos de alto nível, permitindo que os clientes executem tarefas como cadastro de produtos, registro de lotes, consulta de validade e geração de relatórios por meio de chamadas simples e padronizadas.

Impacto Arquitetural

A classe SistemaEstoque estabelece-se como o ponto central de comunicação entre as interfaces da aplicação e as classes internas de negócio, como **Fornecedor**, **Produto**, **Lote** e **Estoque**. Dessa forma, os módulos consumidores deixam de depender diretamente das implementações internas, promovendo maior coesão e reduzindo o acoplamento do sistema.

Vantagens

- Reduz o acoplamento entre os consumidores e as classes do domínio;
- Simplifica a utilização do sistema;
- Padroniza e garante a execução das regras de negócio organizadas na sequência correta;
- Facilita modificações internas no domínio sem quebrar sistemas dependentes.

Possíveis Limitações

- Há o risco de a fachada se transformar em um God Object (objeto onisciente), acumulando responsabilidades excessivas se não houver disciplina na distribuição das subcamadas.
- A fachada pode concentrar responsabilidades excessivas caso novas funcionalidades sejam continuamente adicionadas;

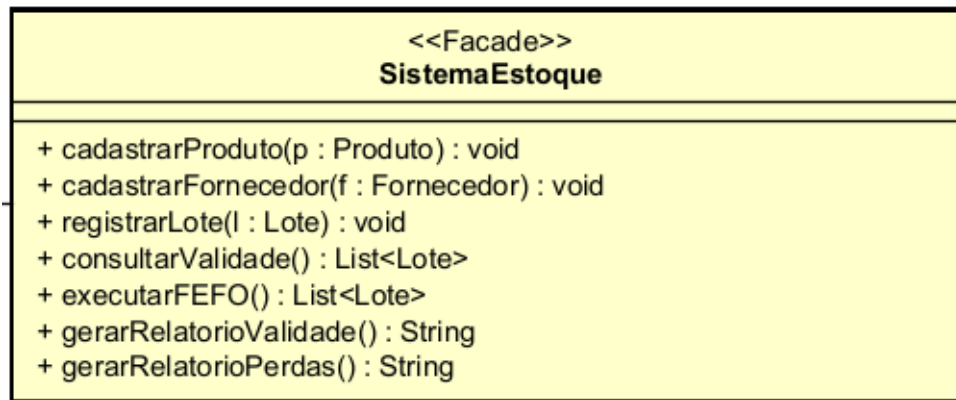


Figura 9: Diagrama:Facade

2.4 Padrão 4: Strategy [Comportamental]

Problema Identificado

O sistema exige suporte a diferentes lógicas e algoritmos logísticos de triagem e vazão de mercadorias perecíveis (como priorizar o vencimento ou a data de entrada). Na abordagem original, estes critérios de negócio estavam engessados em blocos condicionais encadeados no núcleo da classe que manipulava os dados, tornando a manutenção onerosa e ferindo frontalmente o princípio Open/Closed (OCP).

Solução Proposta

Foi aplicado o padrão Strategy por meio da interface *EstrategiaReposicao*, responsável por definir o contrato `ordenarSaida(lotes : List<Lote>) : List<Lote>`. Os algoritmos de reposição foram encapsulados em classes independentes: *FEFOStrategy*, que prioriza os lotes com menor data de vencimento, e *FIFOStrategy*, que prioriza a ordem de entrada dos lotes. A classe *Estoque* mantém uma referência para a estratégia atual por meio do atributo `estrategia : EstrategiaReposicao`, permitindo alterar dinamicamente a lógica de reposição utilizada.

Justificativa Técnica

O padrão substitui estruturas condicionais complexas por composição de objetos. Dessa forma, a classe *Estoque* delega a ordenação dos lotes para a estratégia configurada, permitindo modificar o comportamento de reposição sem alterar a estrutura interna do estoque.

Por que usar o padrão?

Diferentes situações de armazenamento e distribuição de produtos podem exigir critérios distintos de saída. O padrão Strategy permite alternar entre diferentes algoritmos de reposição em tempo de execução, facilitando a adaptação do sistema e a inclusão de novas estratégias sem modificar as classes existentes.

Impacto Arquitetural

Os algoritmos de ordenação foram desacoplados da estrutura responsável pelo armazenamento dos lotes, permitindo que cada estratégia seja desenvolvida, mantida e testada de forma independente. Isso aumenta a coesão das classes e favorece a extensibilidade do sistema.

Vantagens

- Elimina condicionais complexas;
- Torna o código do gerenciador estável e focado na sua responsabilidade primária;
- Concede maleabilidade comportamental ao sistema em tempo de execução;
- Favorece a conformidade com regras de negócio específicas.

Possíveis Limitações

- Aumenta a quantidade de classes no sistema;
- O cliente precisa estar ciente das implementações de estratégia disponíveis para configurá-las corretamente.

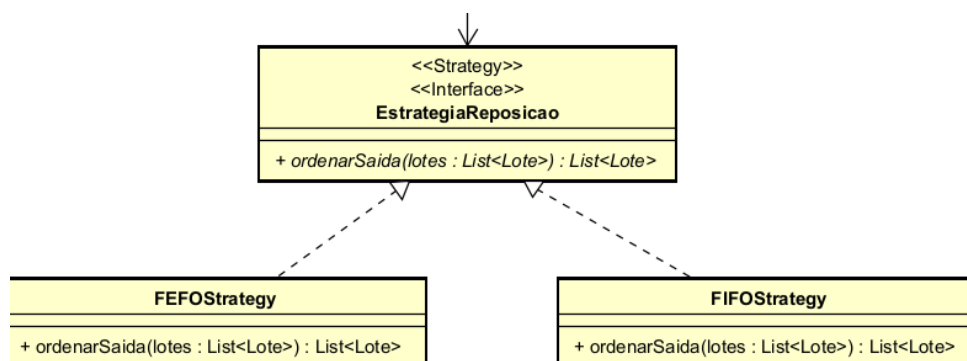


Figura 10: Diagrama: Strategy

2.5 Padrão 5: Template Method [Comportamental]

Problema Identificado

Foi aplicado o padrão Template Method por meio da interface abstrata Relatorio, responsável por definir o método `gerarRelatorio() : String`, que estabelece a sequência de execução para a geração dos documentos. Esse método coordena a chamada das operações protegidas `coletarDados()`, `processarDados()` e `formatar()`. As subclasses concretas `RelatorioValidade` e `RelatorioPerdas` implementam essas etapas variáveis, preservando a estrutura geral do algoritmo definida pela superclasse.

Solução Proposta

Este método orquestra a chamada encadeada das rotinas protegidas e abstratas `#coletarDados()`, `#processarDados()` e `#formatar()`. As subclasses filhas concretas `RelatorioValidade` e `RelatorioPerdas` realizam a herança e estendem estritamente o comportamento variável das etapas, mantendo a sequência principal intocada.

Justificativa Técnica

O padrão Template Method garante a padronização do processo de geração de relatórios por meio da reutilização de uma estrutura comum. Dessa forma, as subclasses podem especializar as etapas de coleta, processamento e formatação dos dados sem modificar o fluxo principal estabelecido pelo método `gerarRelatorio()`.

Por que usar o padrão?

Os relatórios do sistema compartilham um mesmo fluxo de execução, diferenciando-se apenas pelos dados manipulados e pela forma de apresentação das informações. O Template Method permite reutilizar a sequência principal do algoritmo e concentrar nas subclasses apenas as particularidades de cada tipo de relatório.

Impacto Arquitetural

A classe abstrata `Relatorio` passa a definir o fluxo padrão para geração de documentos, enquanto as subclasses `RelatorioValidade` e `RelatorioPerdas` são responsáveis por implementar os comportamentos específicos de cada relatório. Isso promove maior reutilização de código e padronização dos documentos gerados.

Vantagens

- Elimina duplicação de código;
- Garante uma sequência padronizada de execução;
- Facilita a criação de novos relatórios;
- Reduz custos de manutenção ao isolar mudanças no fluxo geral em um único ponto abstrato.

Possíveis Limitações

- Acoplamento via herança: subclasses dependem da estrutura da classe abstrata, dificultando refatorações na hierarquia;
- Apresenta rigidez caso surja um novo modelo de relatório que demande passos completamente divergentes do template pré-estabelecido.

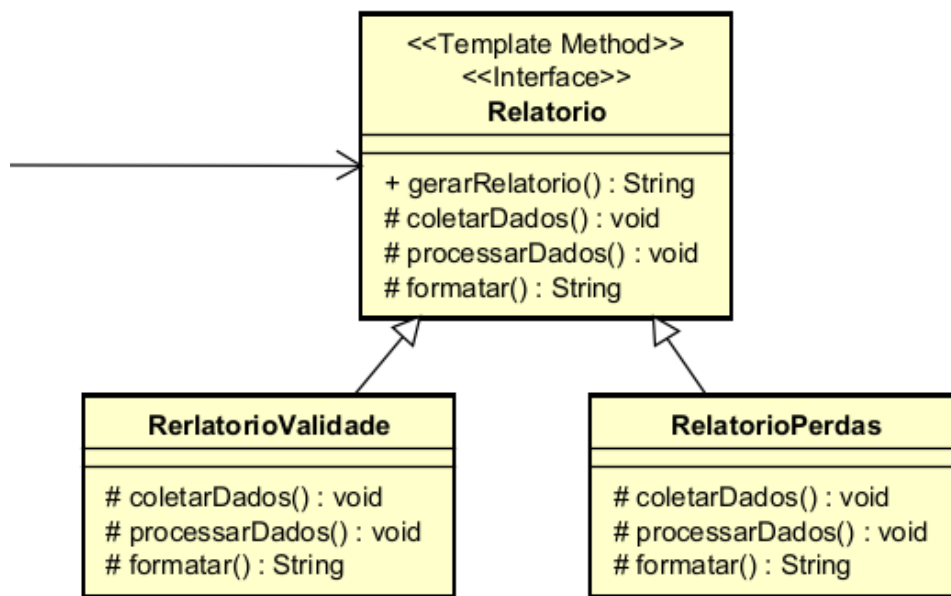


Figura 11: Diagrama:Method